Methods for optimizing the training of Neural Networks

Faculty of Computer Science,
"Alexandru Ioan Cuza" University

October 30, 2026

Backpropagation Recap: Error Propagation

The forward step:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = \sigma\left(z^{(l)}\right)$$

$W^{(l)}$ are weights, $b^{(l)}$ are biases, $\sigma(\cdot)$ is the activation function, $a^{(l-1)}$ are the activations from the last layer. For the backward step, we have:

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}} = \frac{\partial \mathcal{L}}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial z^{(l)}} = \frac{\partial \mathcal{L}}{\partial a^{(l)}} \sigma'\left(z^{(l)}\right)$$

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \sigma'\left(z^{(L)}\right)$$

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l+1)}} \frac{\partial z^{(l+1)}}{\partial a^{(l)}} \sigma'\left(z^{(l)}\right) = \left(W^{(l+1)}\right)^T \delta^{(l+1)} \sigma'\left(z^{(l)}\right)$$

Backpropagation Recap: Chain Rule and Gradients

For a parameter $W^{(l)}$, the loss depends on it through:

$$W^{(l)} \to z^{(l)} \to a^{(l)} \to \cdots \to \mathcal{L}$$

Applying the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial W^{(l)}} = \delta^{(l)} \frac{\partial z^{(l)}}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)}\right)^T$$

The parameter update is:

$$W_{t+1}^{(l)} \leftarrow W_t^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W_t^{(l)}}$$

$$W_{t+1} = W_t - \eta G$$

where η is the learning rate.

Optimizing the parameters of Neural Networks

Optimization problem: train a NN on a dataset D given a cost function L.

$$\min_W L(W) = \frac{1}{N} \sum_{i \in D} L_i(W)$$

Gradient Descent (full batch learning)

$$W_{t+1} = W_t - \eta_t g$$

$$g = \frac{1}{N} \sum_{i \in D} g_i$$

$$g_i = \nabla L_i(W_t)$$

$$W_{t+1} = W_t - \eta_t \frac{1}{N} \sum_{i \in D} \nabla L_i(W_t)$$

Optimizing the parameters of Neural Networks

Online Stochastic Gradient Descent

$$W_{t+1} = W_t - \eta_t \nabla L_{i_t}(W_t)$$

Mini-batch Stochastic Gradient Descent

$$W_{t+1} = W_t - \eta_t \hat{g}_t$$

$$\hat{g}_t = \frac{1}{BS} \sum_{i \in B_t} \nabla L_i(W_t), \quad \text{BS} = \text{batch size}$$

$$W_{t+1} = W_t - \eta_t \left(\frac{1}{BS} \sum_{i \in B_t} \nabla L_i(W_t) \right)$$

Optimizers - SGD

$$W_{t+1} \leftarrow W_t - \eta \hat{g}_t$$

```
def sgd(weights, grads, momentum_buffer_list, weight_decay,
    momentum, lr, nesterov):
    for i, weight in enumerate(weights):
        dW = grads[i]  # gradient for weight on layer i
        weight = weight - lr * dW
        weights[i] = weight
```

Optimizers - SGD + Momentum

$$M_t \leftarrow \mu M_{t-1} + \hat{g}_t$$

$$W_{t+1} \leftarrow W_t - \eta M_t$$

```
def sgd(weights, grads, momentum_buffer_list, weight_decay,
    momentum, lr, nesterov):
    for i, weight in enumerate(weights):
        dW = grads[i]  # gradient for weight on layer i
        if momentum != 0:
            buf = momentum_buffer_list[i]  # get momentum from t-1
            buf = buf * momentum + dW
            momentum_buffer_list[i] = buf  # update momentum buffer
            if nesterov:  # use Nesterov rule
                buf = dW + momentum * buf
            dW = buf  # the gradient becomes the momentum
        weight = weight - lr * dW
        weights[i] = weight
```

Optimizers - SGD + regularization

L2: $W_{t+1} \leftarrow W_t - \eta \hat{g}_t - \eta \lambda W_t$

```
def sgd(weights, grads, momentum_buffer_list, weight_decay,
    momentum, lr, nesterov):
    for i, weight in enumerate(weights):
        dW = grads[i]  # gradient for weight on layer i
        if weight_decay != 0:
            dW = dW + weight * weight_decay
        if momentum != 0:
            ...
        weight = weight - lr * dW
        weights[i] = weight
```

L1: $W_{t+1} \leftarrow W_t - \eta \hat{g}_t - \eta \lambda \, \text{sign}(W_t)$

Optimizers - Adam

Adam is one of the most popular optimizers.

$$M_t \leftarrow \beta_1 M_{t-1} + (1-\beta_1)\hat{g}_t, \quad \text{(from Momentum)}$$

$$v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2)\hat{g}_t^2, \quad \text{(from RMSProp)}$$

$$\hat{m}_t \leftarrow \frac{M_t}{1-\beta_1^t}, \qquad \hat{v}_t \leftarrow \frac{v_t}{1-\beta_2^t}$$

$$W_{t+1} \leftarrow W_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

AdamW:

$$W_{t+1} \leftarrow W_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta\lambda W_t$$

Optimizers - Muon

Muon is a recent state-of-the-art optimizer: link.

$$M_t \leftarrow \mu M_{t-1} + \hat{\mathrm{g}}_t$$

$$O_t \leftarrow \mathrm{NewtonSchulz5}(M_t)$$

$$W_{t+1} \leftarrow W_t - \eta O_t$$

Muon can be used to optimize only 2D weight matrices. For other parameters, AdamW is used instead.

Optimizers - other optimizers

- A visualization of different optimizers: link.

Learning Rate Schedulers

- $W_{t+1} \leftarrow W_t - \eta \hat{g}_t$
- bigger learning rates may cause overshooting;
- smaller learning rates lead to slow convergence.

Learning Rate Schedulers

- Illustration of learning rate policies: link.

Batch Size Schedulers

- $W_{t+1} \leftarrow W_t - \eta \hat{g}_t$
- Decreasing the learning rate is proportional to increasing the batch size: $W_{t+1} = W_t - \eta_t \left(\frac{1}{BS} \sum_{i \in B_t} \nabla L_i(W_t) \right)$
- Batch size scheduling can decrease the training time by more than 30% due to better hardware utilization.
- Batch size schedulers implementation: link.
- Visualization of batch size schedulers: link.

References

1. LeCun, Y. A., Bottou, L., Orr, G. B., and M¨uller, K.-R. (2012). Efficient backprop. In Neural Networks: Tricks of the Trade: Second Edition, pages 9–48. Springer.
2. Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., and Tang, P. T. P. (2017). On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima. In International Conference on Learning Representations.
3. Devarakonda, A., Naumov, M., and Garland, M. (2017). Adabatch: Adaptive batch sizes for training deep neural networks. arXiv preprint arXiv:1712.02029.
4. Smith, S. L., Kindermans, P.-J., Ying, C., and Le, Q. V. (2018). Don't Decay the Learning Rate, Increase the Batch Size. In International Conference on Learning Representations.

References

1. D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings, Y. Bengio and Y. LeCun, Eds., 2015.
2. Foret, P., Kleiner, A., Mobahi, H., and Neyshabur, B. (2021). Sharpness-aware minimization for efficiently improving generalization. In International Conference on Learning Representations.
3. G. Stoica and M. E. Breabăn, "bs-scheduler: A Batch Size Scheduler library compatible with PyTorch DataLoaders," SoftwareX, vol. 30, p. 102162, 2025.
4. G. Stoica, "Increasing the Training Speed with Batch Size Schedulers," 2025 27th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC), Timisoara, Romania, 2025, pp. 540-547.